El Takibi Pro

Webcam görüntüsünden parmak sayısına, parmak sayısından ses kontrolüne

El hareketleriyle bilgisayar sesini yönetmek

Python ile geliştirilen masaüstü görüntü işleme projesi

Kapsam: canlı görüntü, eklem çizimi, açık parmak sayımı ve ses seviyesi kontrolü.

1. Projenin amacı ve gereksinimleri

Kamera görüntüsünü anlaşılır bir kontrol komutuna dönüştürmek

01 Canlı görüntü

Webcamden kareleri alıp arayüzde göstermek.

02 El ve eklem tespiti

Eli algılamak, 21 eklem noktasını ve bağlantılarını çizmek.

03 Parmak sayımı

Açık parmakları hesaplamak; toplamı ekrana yazmak.

04 Ses kontrolü

Tek eldeki 0–5 parmağı %0–100 ses seviyesine dönüştürmek.

2. Kullanılan teknolojiler

Her kütüphane uygulamanın farklı bir görevini üstlenir

Python + Tkinter / ttk

Uygulama mantığı, pencere, düğmeler, mod seçimi ve durum göstergeleri.

OpenCV + MediaPipe

Kameradan görüntü alma, el noktalarını algılama ve görüntü üzerine çizim.

Pillow + threading / queue

Kareleri arayüzde gösterme; takip işlemini ayrı iş parçacığında yürütme.

Pycaw + Windows Core Audio

Varsayılan ses çıkışının seviyesini ve sessiz durumunu değiştirme.

3. Sistemin çalışma akışı

Görüntü işleme ile arayüz birbirinden ayrılır

Webcam → **El tespiti** → **Parmak sayımı** → **Arayüz** → **Ses kontrolü**

Worker.run(): kareyi al → RGB'ye çevir → modeli çalıştır → say → çiz

Worker.send() → sınırlı kuyruk → Studio.poll() → Studio.render()

Ses kontrolü modu açıksa kararlı parmak sayısı Windows ses seviyesine uygulanır.

4. Kamera ve model fonksiyonları

camera.py • tracking.py • main.py

open_camera(cv2, index, backend, stop)

Kamerayı açar; Windows'ta otomatik modda sürücüleri sırayla dener. İlk geçerli kareyi döndürür.

ensure_model(path)

Model mevcutsa kullanır; yoksa indirir. İndirme tamamlanınca geçici dosyayı asıl dosyaya taşır.

Worker.run()

Kamerayı ve modeli yönetir; her karede el tespiti ve parmak sayımını yürütür.

HandLandmarker.detect_for_video(image, timestamp)

MediaPipe çağrısıdır. Görüntüdeki ellerin eklem noktalarını ve 3B dünya koordinatlarını üretir.

5. El eklemleri ekrana nasıl çizilir?

main.py → Worker.run() içindeki çizim adımları

Koordinat dönüşümü

Normalize noktalar piksele çevrilir: x_piksel = int(x × genişlik), y_piksel = int(y × yükseklik).

cv2.line() ve cv2.circle()

CONNECTIONS listesindeki eklemler çizgilerle bağlanır; her eklem noktası daireyle gösterilir.

cv2.putText()

Her elin hesaplanan parmak sayısını bilek noktasının yanına yazar.

Studio.render()

Pillow ile kareyi önizlemeye yerleştirir; toplam parmak sayısını, el sayısını ve FPS bilgisini günceller.

6. Açık parmak sayımı nasıl yapılır?

tracking.py → count_fingers(landmarks, threshold=155)

angle(a, b, c)

Üç noktanın B ekleminde oluşturduğu açıyı vektörler ve kosinüs bağıntısıyla hesaplar.

distance(a, b)

İki 3B nokta arasındaki Öklid uzaklığını hesaplar; parmak ve avuç ölçümlerinde kullanılır.

count_fingers()

Dört parmakta iki eklem açısını ve bileğe uzaklığı kontrol eder. Başparmak için ayrı açılma koşulları uygular.

Çıktı: el başına 0–5

Açık kabul edilen parmak durumlarını toplar. Bu geometrik yaklaşım yaklaşık sayım yapar.

7. Sayımın dengelenmesi

Kısa süreli algılama değişimlerinin etkisini azaltmak

deque(maxlen=smoothing)

Son birkaç karenin toplam parmak sayısını tutar. Varsayılan pencere 7 karedir.

Counter(history).most_common(1)

Pencerede en sık görülen toplamı seçer; ekranda daha kararlı bir sayı gösterir.

El sayısı değişirse geçmiş sıfırlanır

Hiç el görünmüyorsa sonuç None olur. Bu durum, görünen kapalı yumruktan (0 parmak) farklıdır.

Ses için ek bekleme: 0,6 saniye

Dengelenmiş sayı tek elde sabit kalınca ses uygulanır. Dengeleme toplam tepki süresine ek gecikme katabilir.

8. Parmak sayısından ses seviyesine

Ses kontrolü modunda tek el kullanılır

0 parmak	1 parmak	2 parmak	3 parmak	4 parmak	5 parmak
%0	%20	%40	%60	%80	%100

Ses yüzdesi = açık parmak sayısı × 20

0 parmak sesi sıfırlar ve sessize alır. Pozitif seviye sessizi kaldırır.

El yoksa veya iki el varsa ses değişmez. Takip durunca son seviye korunur.

9. Ses kontrolünün önemli fonksiyonları

volume_control.py → Windows'a ses komutu gönderme

windows_volume_setter()

Windows ve bağımlılık kontrolünü yapar. AudioUtilities.GetSpeakers() ile varsayılan çıkışı alır.

FingerVolume.update(count, hands, now)

Tek el, geçerli sayı ve 0,6 saniyelik sabit poz koşullarını kontrol eder. Yeni seviyeyi bir kez uygular.

set_percent(percent)

İç fonksiyon: SetMasterVolumeLevelScalar(percent / 100, None) ile seviyeyi ayarlar; SetMute() ile sessizi yönetir.

Hata davranışı

Ses aygıtı hatasında kullanıcıya mesaj gösterilir ve ses kontrolü durur. Kamera takibi devam eder.

10. Arayüzü yöneten önemli metotlar

main.py → Studio sınıfı

Studio.start() / Studio.stop()

Ayarları doğrular, takip iş parçacığını başlatır veya durdurma işaretini gönderir.

Studio.poll() / Studio.render()

Kuyruktaki durum ve kareleri işler; canlı görüntüyü, sayımı ve seçilen modun bilgilerini günceller.

Studio.save_image()

Kullanıcının seçtiği konuma o anki görüntüyü PNG olarak kaydeder.

Studio.close() / Worker.send()

Kapanışı yönetir; kareleri ve durum mesajlarını iş parçacıkları arasında taşır.

11. Sunumda gösterilecek demo

Windows çalışma izni ve bağımlılıklar hazır olduktan sonra

1 Kamerayı başlat

Kamera numarasını seç; canlı görüntüyü ve eklem çizgilerini göster.

2 Parmak sayımını göster

Önce tek elde, sonra iki elde açık parmak sayısını değiştir; ekrandaki toplamı izle.

3 Ses kontrolü moduna geç

Takibi durdur, modu seç ve yeniden başlat. 1, 2 ve 3 parmağı sabit tutarak %20, %40, %60 seviyelerini göster.

4 El kaybını ve durdurmayı göster

Elini kadrajdan çıkar: ses sabit kalmalı. Durdur düğmesiyle kamerayı serbest bırak.

12. Test sonuçları ve mevcut sınırlar

Kod düzeyindeki doğrulama ile gerçek cihaz testi ayrıdır

7 otomatik test başarılı

Açı hesabı, bozuk el verisi, kamera sürücü geçişi ve ses kontrolü durumları test edildi.

Donanım doğrulaması bekliyor

Canlı webcam, gerçek ses çıkışı ve uçtan uca performans bu çalışmada doğrulanmadı.

Mevcut engel

Windows grup ilkesi projenin Python dosyasını engelledi. Çalıştırma izni sağlanmalı; Kurulum.bat ile ses bağımlılığı kurulmalı.

Sayım; örtüşen eller, yan görünüş ve kıvrık parmaklarda yanılabilir. Ölçülmüş doğruluk oranı veya FPS sonucu bulunmuyor.

13. Sonuç ve kaynaklar

Kamera görüntüsünden temassız bir kontrol arayüzüne

Proje; görüntü alma, el tespiti, geometrik parmak sayımı ve işletim sistemi ses kontrolünü tek uygulamada birleştirir.

İncelenen proje dosyaları

main.py • camera.py • tracking.py • volume_control.py
tests/test_core.py • tests/test_volume.py • requirements.txt

Ses API başvurusu

Pycaw — AudioDevice.EndpointVolume ve ses seviyesi API’si

https://andremiras.github.io/pycaw/api/index.html